

GLOBAL SOLUTION SPACE CONNECT

Sistema de Monitoramento de Colônia Espacial em Marte

CREW MANIFEST

Calebe Gonçalves Garcia de Souza
RM568743

Lucas Ribeiro Gesini
RM569383

Raphael de Freitas Silva
RM570089

Filipe Souza Nascimento
RM573758

Paulo Henrique Gonçalves Bueno
RM570456

MISSÃO ATUAL: EXPANSÃO DA COLÔNIA EM MARTE | TURMA 2026 | 1CCOC

Sumário

1. Visão Geral do Projeto

2. Arquitetura do Sistema

3. Módulos do Sistema

4. Dados da Missão

5. Estruturas de Dados

6. Lógica de Diagnóstico e Alertas

7. Simulação e Previsões

8. Conclusão

1. Visão Geral do Projeto



O **Global Solution Space Connect** é um sistema de monitoramento em tempo real para uma colônia espacial em Marte. Desenvolvido em Python como solução para o desafio acadêmico da FIAP, o projeto simula o gerenciamento operacional de uma base marciana, abrangendo controle energético, telemetria ambiental, comunicações e suporte à vida.

Missão	Mars Colony Expansion
Status	ALERTA — Radiação elevada
Reserva de Energia	48%
Temperatura	23°C (média)
Radiação	88 mSv/h — ALTA
Comunicação	78% — Instável

2. Arquitetura do Sistema



O sistema adota uma arquitetura em camadas com separação clara de responsabilidades. O ponto de entrada (**main.py**) coordena o menu principal e delega cada operação a um módulo especializado. A camada de dados (**monitoramento.py**) é responsável pelo carregamento do arquivo JSON de missão.

Arquivo	Responsabilidade
main.py	Menu principal e orquestração dos módulos
monitoramento.py	Carregamento e parsing do JSON de missão
modulos/diagnostico.py	Análise de status e exibição do diagnóstico
modulos/alertas.py	Fila de alertas com deque
modulos/recomendacoes.py	Geração de recomendações operacionais
modulos/eventos.py	Pilha de eventos da missão
modulos/hierarquia.py	Exibição da hierarquia de sistemas
modulos/simulacao.py	Simulação por ciclos com variação aleatória
modulos/previsao.py	Previsão de reserva energética
modulos/tendencia.py	Análise de tendência de consumo
modulos/analise_matriz.py	Cálculo de variação de energia e temperatura
modulos/inconsistencias.py	Deteção de dados inconsistentes
modulos/graficos.py	Dashboard gráfico com matplotlib

3. Módulos do Sistema

3.1 Diagnóstico (diagnostico.py)

- A função **analisar_missao()** avalia energia, radiação e qualidade de comunicação em combinação, gerando alertas na fila conforme os limiares definidos.
- A função **exibir_diagnostico()** imprime um painel formatado com status, reserva atual, previsão e tendência.

Código:

```
def analisar_missao(dados, fila_alertas):  
    energia = dados["reserva_energia"]  
    radiacao = dados["radiacao"]  
    comunicacao = dados["qualidade_comunicacao"]  
    status = "NORMAL"  
    if energia < 35 and comunicacao < 50 and radiacao > 70:  
        status = "EMERGÊNCIA TOTAL"  
        fila_alertas.append(("CRÍTICO", "Colapso múltiplo"))  
    elif radiacao > 80:  
        status = "ALERTA"  
        fila_alertas.append(("ALERTA", "Radiação elevada"))  
    return status
```

3.2 Alertas (alertas.py)

- Utiliza **collections.deque** como fila FIFO de alertas.
- Cada alerta é uma tupla (**nível, mensagem**) onde o nível pode ser CRÍTICO, ALERTA ou INFO.

Código:

```
from collections import deque  
  
def criar_fila_alertas():  
    return deque()  
  
def exibir_alertas(fila_alertas):  
    for nivel, msg in fila_alertas:  
        print(f"[{nivel}] {msg}")
```

3.3 Previsão de Energia (previsao.py)

- A fórmula utilizada é: $\text{previsão} = \text{reserva_atual} - (\text{média_consumo} / 10)$.

Código:

```
def prever_energia(dados):  
    consumo = dados["energia_consumida"]  
    media = sum(consumo[-3:]) / 3  
    previsao = dados["reserva_energia"] - (media / 10)  
    return previsao
```

3.4 Tendência (tendencia.py)

Código:

```
def analisar_tendencia(dados):  
    consumo = dados["energia_consumida"]  
    if consumo[-1] > consumo[-2]: return "AUMENTO"  
    elif consumo[-1] < consumo[-2]: return "REDUÇÃO"  
    else: return "ESTÁVEL"
```

3.5 Simulação (simulacao.py)

Código:

```
def atualizar_dados(dados):  
    dados["reserva_energia"] += random.randint(-3, 3)  
    dados["radiacao"] += random.randint(-2, 2)  
    dados["qualidade_comunicacao"] += random.randint(-5, 5)  
    # Normaliza entre 0 e 100  
    dados["reserva_energia"] = max(0, min(100, dados["reserva_energia"]))
```

3.6 Inconsistências (inconsistencias.py)

- Detecta contradições nos dados: módulo de comunicação desligado com sinal alto, energia fora do intervalo 0–100, radiação extrema sem suporte à vida.

Código:

```
if comunicacao_status == 0 and qualidade_comunicacao > 70:

    inconsistencias.append(

        ("CRÍTICO", "Comunicação desligada com sinal alto"))

if radiacao > 85 and dados["modulos"]["suporte_vida"]:

    inconsistencias.append(

        ("ALERTA", "Radiação extrema sem proteção"))
```

4. Dados da Missão

O arquivo **dados_missao.json** serve como fonte de verdade do sistema. Ele armazena o estado atual de todos os sensores, históricos de série temporal e a hierarquia de sistemas. A estrutura JSON foi escolhida por ser legível, facilmente editável e nativamente suportada em Python.

4.1 Status dos Módulos

Módulo	Status
Suporte_Vida	ONLINE
Energia	ONLINE
Comunicacao	OFFLINE
Habitat	ONLINE
Laboratorio	ONLINE
Armazenamento	ONLINE

4.2 Séries Temporais

Série	Valores (8 ciclos)
Energia Gerada (kWh)	72, 75, 70, 68, 74, 71, 69, 73
Energia Consumida (kWh)	65, 67, 70, 72, 75, 77, 79, 82
Temperatura (°C)	22, 23, 24, 25, 24, 23, 22, 21
Histórico Radiação	60, 62, 58, 64, 66, 70, 74, 78
Histórico Reserva (%)	70, 68, 65, 61, 58, 55, 52, 49

5. Estruturas de Dados



5.1 Fila de Alertas — deque

A fila de alertas usa `collections.deque`, estrutura FIFO (First-In, First-Out) otimizada para inserções e remoções nas extremidades em $O(1)$. Isso garante que alertas críticos gerados primeiro sejam processados primeiro.

```
# Criação e uso da fila FIFO
from collections import deque

fila = deque()

fila.append(("CRÍTICO", "Energia crítica")) # enqueue –  $O(1)$ 
fila.append(("ALERTA", "Radiação elevada"))
nivel, msg = fila.popleft() # dequeue –  $O(1)$ 
```

5.2 Pilha de Eventos — list (LIFO)

```
# Pilha de eventos – últimos 8 eventos em ordem reversa
for i, evento in enumerate(list(eventos)[-8:::-1], start=1):
    tabela.append([i, evento])
```

5.3 Dicionário Aninhado — Hierarquia

```
"hierarquia": {
  "Energia": { "Solar": "Ativo", "Baterias": "Operacional", "Reserva": "48%" },
  "Habitat": { "Oxigenio": "Estavel", "Temperatura": "Controlada", "Comunicacao": "Instavel" },
  "Pesquisa": { "Laboratorio": "Operacional", "Sensores": "Ativos" }
}
```

6. Lógica de Diagnóstico e Alertas

Condição	Status	Alerta Gerado
Energia<35 E Comunic<50 E Radiação>70	EMERGÊNCIA TOTAL	Colapso múltiplo
Energia<35 E Comunicação<50	CRÍTICO	Energia e comunicação baixas
Radiação > 80	ALERTA	Radiação elevada
Módulo inoperante	CRÍTICO	Módulo {nome} inoperante
Todas condições normais	NORMAL	— nenhum —

6.1 Estado Atual da Missão

Com base nos dados carregados do `dados_missao.json`, o diagnóstico atual é:

- [ALERTA] Radiação elevada — 88 mSv/h
- [CRÍTICO] Módulo comunicacao inoperante
- [CRÍTICO] Comunicação desligada com sinal alto (78%)
- ✓ Suporte à vida: ONLINE
- ✓ Habitat: ONLINE
- ✓ Energia solar: ATIVO

6.2 Recomendações Geradas

7. Simulação e Previsões



O módulo de simulação (`simulacao.py`) executa N ciclos consecutivos, aplicando variações pseudo-aleatórias nos sensores a cada iteração. Isso permite validar o comportamento do sistema diagnóstico sob diferentes cenários operacionais sem necessidade de hardware real.

7.1 Previsão Energética

```
consumo_recente = [79, 79, 82] # últimos 3 valores
media = sum(consumo_recente) / 3 # = 80.0
reserva_atual = 48
previsao = 48 - (80.0 / 10) # = 40.0%
tendencia = "AUMENTO" # pois 82 > 79
```

Com consumo em tendência de AUMENTO e reserva prevista em ~40%, o sistema recomenda iniciar medidas preventivas de economia antes de atingir o limiar crítico de 35%.

7.2 Matriz de Telemetria

Variação de Energia (último ciclo)	73 - 82 = -9 kWh
Média de Temperatura	23.0 °C
Tendência de Consumo	AUMENTO
Previsão de Reserva	40.0%

7.3 Dashboard Gráfico

O módulo `graficos.py` utiliza `matplotlib` para gerar um painel com 4 subgráficos: Energia (gerada vs consumida), Temperatura, Radiação e Reserva Energética. O dashboard é acessível pela opção 5 do menu principal.

8. Conclusão

O **Global Solution Space Connect** demonstra a aplicação prática de conceitos fundamentais de Ciências da Computação em um cenário de alta complexidade e criticidade. A escolha das estruturas de dados (deque para alertas, list como pilha para eventos, dicionários aninhados para hierarquia) foi motivada por eficiência e adequação semântica a cada caso de uso.

Integrante	RM
Calebe Gonçalves Garcia de Souza	RM568743
Filipe Souza Nascimento	RM573758
Lucas Ribeiro Gesini	RM569383
Paulo Henrique Gonçalves Bueno	RM570456
Raphael de Freitas Silva	RM570089